Milestone 2: Formula 1 Knowledge Graph

Deadline: Tuesday 17th of November 2026 at 23:59

Introduction

After completing Milestone 1, you have a solid understanding of the Formula 1 dataset: its relational structure, the links between its tables, and the risk of data leakage.

Building on this foundation, Milestone 2 focuses on constructing a Knowledge Graph (KG) in Neo4j that semantically represents the Formula 1 domain. The objective is to transform the relational tables into an interconnected graph of drivers, constructors, races, circuits, seasons, and the race-specific facts that connect them. This KG will serve as the foundation for Milestone 3.

A key design idea in this milestone is the difference between entities and facts. A driver's name or a circuit's country rarely changes, so it belongs on the entity node. A driver's constructor, grid slot, or finishing position changes from race to race, so it belongs on a fact node (Result, Qualifying, PitStop) that links the driver to that specific race.

The Dataset

You will use the same Formula 1 World Championship dataset from Milestone 1. Nine of its 14 tables are used to build the KG.

File	Rows	Used for
drivers.csv	861	Driver nodes
constructors.csv	212	Constructor nodes
circuits.csv	77	Circuit nodes
seasons.csv	75	Season nodes
status.csv	139	Status nodes
races.csv	1,125	Race nodes
results.csv	26,759	Result fact nodes
qualifying.csv	10,494	Qualifying fact nodes
pit_stops.csv	11,371	PitStop fact nodes

The remaining tables (lap_times, driver_standings, constructor_standings, constructor_results, sprint_results) are not part of the KG in this milestone. Consequently, all points totals in this milestone refer to Grand Prix results only, excluding sprints.

Missing values (the literal \N in the CSVs, and the empty q2 and q3 cells in qualifying.csv) must not be stored as property values; omit the property instead.

Milestone 2 Requirements

This milestone requires you to construct a Knowledge Graph in Neo4j that represents the Formula 1 domain according to the schema below, then query it with Cypher.

- **Knowledge Graph Construction.** Implement a Python script (using the Neo4j driver) that reads the CSV files and builds the KG according to the exact schema below. Parent nodes (Driver, Constructor, Circuit, Season, Status, Race) must exist before the fact nodes that link to them. Re-running the script must not create duplicate nodes or relationships.
- **Schema Compliance.** You must strictly follow the provided schema. You may not:
  - change the label of any node (e.g., rename “Driver” to “Racer”);
  - change the type of any relationship (e.g., rename “:BY_DRIVER” to “:DRIVEN_BY”);
  - remove or rename any property specified for a node;
  - remove any node label or relationship type from the schema.
- **Cypher Query Development.** Write Cypher queries that answer the verification questions below. They range from simple filtering to multi-hop traversals across entity and fact nodes, and require care with relationship directions, missing values, and aggregation.

The Knowledge Graph Schema

1. Nodes

Entity nodes:

- `Driver`: driverId (unique identifier), driverRef, number, code, forename, surname, dob, nationality
- `Constructor`: constructorId (unique identifier), constructorRef, constructor_name, constructor_nationality
- `Circuit`: circuitId (unique identifier), circuitRef, circuit_name, circuit_loc, circuit_country, lat, lng, alt
- `Season`: year (unique identifier)
- `Status`: statusId (unique identifier), status
- `Race`: raceId (unique identifier), year, round, name, date, time

Fact nodes:

- `Result`: resultId (unique identifier), number, grid, position, positionText, positionOrder, points, laps, time, milliseconds, fastestLap, rank, fastestLapTime, fastestLapSpeed
- `Qualifying`: qualifyId (unique identifier), number, position, q1, q2, q3
- `PitStop`: eventId (unique identifier, built as raceId:driverId:stop), stop, lap, time, duration, milliseconds

Property types. All identifiers except eventId and all whole-number values (including year, round, number, grid, position, positionOrder, laps, milliseconds, fastestLap, rank, stop, lap, and alt) must be integers. points, lat, lng, and fastestLapSpeed must be floats. Driver.dob and Race.date must be Neo4j dates. All other properties, including times such as q1, fastestLapTime, and duration, must be strings.

2. Relationships

- `(Race) -[:AT_CIRCUIT]-> (Circuit)`
- `(Race) -[:IN_SEASON]-> (Season)`
- `(Result) -[:FOR_RACE]-> (Race)`
- `(Result) -[:BY_DRIVER]-> (Driver)`
- `(Result) -[:FOR_CONSTRUCTOR]-> (Constructor)`
- `(Result) -[:HAS_STATUS]-> (Status)`
- `(Qualifying) -[:FOR_RACE]-> (Race)`
- `(Qualifying) -[:BY_DRIVER]-> (Driver)`
- `(Qualifying) -[:FOR_CONSTRUCTOR]-> (Constructor)`
- `(PitStop) -[:FOR_RACE]-> (Race)`
- `(PitStop) -[:BY_DRIVER]-> (Driver)`

The graph should have 51,113 nodes and 163,510 relationships.

Knowledge Graph Verification Queries

NOTE ON VERIFICATION: These are example questions for initial structure confirmation. Create queries that answer the following questions. The final assessment of the graph's design and robustness will involve a separate **set of private queries** to ensure the graph is flexible and not hardcoded to these specific examples.

Column names. Each question ends with the columns your query must return. Give every column exactly that name using AS, because the autograder finds your answers by column name. A driver column holds forename + ' ' + surname (e.g., Lewis Hamilton), and a constructor column holds constructor_name. Decimal values may be rounded, but to no fewer than 2 decimal places.

1. For the 2021 season, return each driver's total Grand Prix points and number of wins, sorted by points (descending) then wins (descending). A win is a position of 1.
    - Return: Driver, Points, Wins.

2. For 2014–2021, return each constructor's reliability: the share of its race entries with a classified finish (status "Finished" or starting with "+"). Include only constructors with at least 100 entries. Sort by reliability (descending).
    - Return: Constructor, Entries, Classified (the number of entries with a classified finish), Reliability (classified / entries, as a value from 0 to 1, not a percentage).

3. Compare Lewis Hamilton (driverId 1) and Valtteri Bottas (driverId 822) in races where they drove for the same constructor. We need the constructor, the number of races they shared, and how many times each finished ahead (by positionOrder).
    - Return: Constructor, Shared_Races, Hamilton_Ahead, Bottas_Ahead.

4. At Circuit de Monaco (circuitId 6), for races before 2020-01-01, return drivers with at least 5 starts, their average finishing position (positionOrder), and their number of wins. Sort by average finish (ascending), then wins (descending). A start is a race entry whose status is not "Did not qualify", "Did not prequalify", or "Withdrew". Note that a driver can only have 1 start per race, take care that a shared drive can give a driver more than one Result in the same race. Calculate the average finish and wins from all of the driver's remaining Result nodes.
    - Return: Driver, Starts, Avg_Finish, Wins.
5. A driver's primary constructor in a season is the constructor with the most race entries for that driver in that season. Return the number of drivers whose primary constructor in 2021 differs from 2020, and their names. Compare only drivers with race entries in both seasons. If two constructors tie for a driver's most entries in a season, use the lower constructorId. Compare constructors by constructorId, so a renamed team (e.g., Renault to Alpine) counts as a change.
    - Return one row: Drivers_Changed (the number of drivers), Drivers (a list of their full names).

Deliverables

You are required to submit the following by filling out the form https://forms.gle/KcqJKdVqubE92xRT7:

1. **Create_kg.py**
    - Python script that creates the Knowledge Graph in Neo4j.
    - Should read the CSV files located in the same directory.
    - Should follow the exact schema specification.
    - The script should use the config.txt file with Neo4j credentials. Not doing so might result in 0. The script must read the configuration in exactly the format shown in config.txt below (URI, USERNAME, PASSWORD). Failing to do so will result in 0. Read each value by its key, not by its line number, because the config.txt used for grading also has a DATABASE line. Do not hardcode a database name such as neo4j in the script.
2. **config.txt**
    - URI=neo4j://localhost:7687
    - USERNAME=neo4j
    - PASSWORD=your_password
3. **queries.txt**
    - A text file containing the queries for the above questions with their numbers. Save it with UTF-8 encoding.
    - Each query must be written in exactly this format: //1. Query 1, then MATCH (...), then RETURN (...), as shown below. Do not use variants such as 1-query. This is the only way the autograder can extract your queries.
    - The queries must be in the correct order (Query 1 to Query 5), and the results of each query must be sorted as stated in the verification queries section.

```
//1. Query 1
 MATCH (...)
 RETURN (...)

 //2. Query 2
 MATCH (...)
 RETURN (...)

 Etc..
```

Submission link: https://forms.gle/pkBj32QgbiVpSz8x8